目　录

第一部分　原型设计的工程本质

1. 原型到底在解决什么问题
2. 为什么需要工程化方法
3. 阶段总览

第二部分　阶段一：原始资料整理

4. 方法
5. 输出与质量门禁

第三部分　阶段二：事实层固化

6. 为什么需要事实层
7. 方法
8. 输出与质量门禁
9. 常见陷阱

第四部分　阶段三：页面结构设计

10. 信息架构先于页面设计
11. 页面职责设计
12. 数据分层
13. 状态完备性
14. 流程设计
15. 输出与质量门禁

第五部分　阶段四：可交互原型实现

16. 原型的技术标准
17. 架构策略
18. 代码组织
19. 增量迭代策略
20. 输出与质量门禁

第六部分　阶段五：评审与修正

21. 评审准备
22. 评审中的记录方法
23. 评审后修正的分层执行

第七部分　高级工程实践

24. 跨页一致性管理
25. 多端协同设计
26. AI 辅助原型实现的注意事项
27. 持续验证

第八部分　评审检查清单

28. 页面级检查

29. 流程级检查
30. 系统级检查

附录

A. 页面设计文档模板
B. 评审走查清单模板
C. 修改安全检查清单

原型设计工程实践指南

> **适用读者**：产品经理、研发经理、交互设计师——任何负责将业务需求转化为可演示、可评审交互原型的技术人员。
>
> **定位**：不是某个具体项目的回顾，而是从大量原型设计实战中提炼的、可复用的通用工程方法论。本指南覆盖从原始资料到可运行原型的全流程，并假设你可能借助 AI Agent 辅助完成部分工作。

第一部分　原型设计的工程本质

1. 原型到底在解决什么问题

原型不是美工作品，不是技术验证，不是文档的可视化翻译。原型服务于一个具体目标：**让评审方在有限时间内做出判断——这个产品的核心交互、关键页面关系和主要业务规则是否合理。**

如果原型不能帮助评审方做判断，无论它多漂亮、多完整，都是失败的。

原型设计真正困难的不是画页面，而是画页面之前的三个判断：

1. **这页围绕什么对象展开？** ——中心错了，所有字段都跟着错。
2. **这页和其他页面是什么关系？** ——关系错了，用户在流程中会迷路。
3. **这页在非理想状态下长什么样？** ——只设计了成功态的原型，评审方无法判断产品是否真正可用。

2. 为什么需要工程化方法

没有工程化方法的原型设计，常见以下失败模式：

失败模式	根本原因	后果
画了大量页面但讲不清关系	跳过了信息架构设计	评审方看不出业务流程
评审时被问到边界问题答不上来	只做了成功态	信任度崩塌
原型和已确认需求不一致	事实层没有固化	评审成了需求重新确认会
改一处发现牵连十处	没有数据分层和术语统一	返工成本高于重做
AI 辅助生成的页面部分含有臆造内容	未确认事实被当成已定结论	评审方看到的"产品方案"实际是 AI 的猜测

工程化方法的核心目的是：**用最小的返工代价，在评审节点交付一个能让评审方做出判断的原型。**

3. 阶段总览

一个完整的原型设计项目通常经历五个阶段，每个阶段有明确的输入、输出和质量门禁：

```
原始资料整理          事实层固化          页面结构设计
──────────── → ──────────── → ────────────
收集、标注、          提炼对象、          信息架构、
识别矛盾            固化结论            页面职责

    → 可交互原型实现 → 评审与修正
      ──────────    ──────────
      共享状态、      走查验证、
      增量迭代        分层修正
```

铁律：跳过前一阶段的输出直接进入下一阶段，是返工的最大来源。事实层不固化就画页面，一定会导致边画边猜业务规则。页面结构不定义就写代码，一定会导致页面之间关系混乱。

第二部分　阶段一：原始资料整理

4. 方法

4.1 收集不合并

把所有上游材料收到一个目录下——需求文档、会议记录、竞品截图、Excel 功能清单、PPT 方案、口头沟通记录、微信截图。第一步只做收集，绝不合并和整理。

4.2 标注来源与日期

每份材料记录来源人和日期。当材料之间出现矛盾时，来源和日期是唯一的仲裁依据。"小张说的"vs"李总说的"、"3 月 15 日说的"vs"3 月 22 日说的"——这些元数据后续会反复用到。

4.3 三分法标注

对每份材料的内容做三分类标注：

分类	定义	处理方式
事实	被项目方明确确认的业务规则或约束	进入事实层
意见	某个人的倾向性看法	标注来源，不直接当结论
猜测	没有被任何人确认的推测	标注"待确认"，绝不默认成立

特别警告：AI 辅助整理材料时，极容易把"意见"和"猜测"升格为"事实"。任何 AI 产出的整理结果，都需要人类用这个三分法重新校验。

4.4 保持原始资料不变

整理后的结论写在独立文档里，原始材料保持原样。原始资料是仲裁依据，不能被修改。

5. 输出与质量门禁

输出：原始资料目录（含来源、日期）+ 初步矛盾项清单

门禁：能在 30 秒内回答"当前有多少份原始资料、最关键的矛盾点在哪里"。

第三部分　阶段二：事实层固化

6. 为什么需要事实层

原型不是从零创作，而是从已确认的事实出发。如果事实层不固化，原型设计者（无论是人类还是 AI）就会一边猜业务规则一边画页面。

事实层的本质是一张网：业务对象是节点，对象之间的关系和约束是边，业务规则是这张网上的标注。原型的每一个页面都必须能在这张网上找到自己的位置。

7. 方法

7.1 提炼业务对象

从原始资料中挖掘核心业务对象，理清它们之间的关系：

```
[教室] 1:N [终端]     一个教室包含多个终端
[终端] 1:N [桌面]     一个终端上有多个桌面
[终端] → [母机]       母机是具有管理权限的特殊终端
[任务] → [终端集合]   任务作用于一组终端
```

关键是区分"所属关系"（终端属于教室）、"引用关系"（任务引用了终端列表）和"状态关系"（终端当前处于"部署中"状态）。

7.2 提炼稳定结论

把已被项目方确认的业务规则逐条写出：

```
- 本地完成不依赖服务器同步—服务器同步失败不推翻本地已完成结果。
- 终端用途是可扩展枚举字段—不限于"教师终端"和"学生终端"。
- 批量任务必须区分三种终端：受管集合、本次勾选终端、保持现状终端。
```

每条结论必须附带来源（"PRD 第 X 节""3/22 会议确认"）。

7.3 区分跨页结论和单页结论

类型	定义	示例
跨页结论	约束多个页面的字段口径和数据层次	"配置指已持久化的业务值，设置指本次任务参数"
单页结论	只约束一个页面的布局和状态	"TERM-04 初始页把 MAC 放在第一列"

跨页结论必须优先于单页设计。 如果跨页口径不统一，每个页面各自发明一套用词，最终无法整合。

7.4 分文档管理

每个主题的结论独立成文，不要把所有结论堆在一个文档里。推荐结构：

```
05_事实层/
├── 事实索引.md              # 链接所有事实文档
├── 终端侧跨页结论.md        # 终端侧所有页面共享的口径
├── 平台侧跨页结论.md        # 平台侧所有页面共享的口径
├── 跨端结论.md              # 终端和平台之间的协同规则
└── 术语表.md                # 全局术语定义
```

7.5 建立正式基线

当需求清单、业务对象关系图和 PRD 被项目方确认后，复制到独立基线目录，锁定为正式基线。

基线的特殊地位：基线文件不直接修改。所有后续修改在派生文档中进行。如需修改基线本身，必须走"新基线确认"流程。

8. 输出与质量门禁

输出：业务对象关系图 + 正式基线文档 + 事实结论文档（分文件）+ 事实索引

门禁：

1. 每条事实结论都能追溯到原始资料或项目方确认。
2. 不同事实文档之间没有矛盾。
3. 能在 30 秒内找到任意一个业务规则的出处。

9. 常见陷阱

- **把推测写成事实**。项目方没有明确确认的内容，标注"待确认"——永远不要默认成立。
- **事实太抽象**。写了"字段分层"但没说具体哪些页面、哪些字段。事实必须具体到可验证。
- **事实和页面脱钩**。改了事实但页面没跟着改，或者页面改了但事实没更新。事实文档和页面设计之间必须建立引用链。
- **AI 静默补全**。让 AI 辅助整理事实时，AI 会倾向于把"大致合理的猜测"直接写成结论。人类必须逐条核验每条 AI 产出的"事实"是否真正有来源。

第四部分　阶段三：页面结构设计

10. 信息架构先于页面设计

在画任何页面之前，先回答以下问题：

1. 整个产品有多少独立页面或页面组。
2. 用户从哪进入、按什么路径走完、走完后回到哪里。

3. 哪些页面共享相同的业务字段和数据来源。
4. 哪些页面之间是互斥关系（同一时间只能出现一个）。
5. 如果产品涉及多端（管理端 + 终端），各端之间的信息同步关系是什么。

这些问题的答案就是**信息架构总纲**。总纲不需要包含页面细节，但它是所有页面设计的基座。

11. 页面职责设计

11.1 对象·动作·范围三分离

每个页面必须先明确三个问题，然后再谈布局：

问题	示例
这页围绕什么对象？	当前教室的终端列表
这页承接什么动作？	选择本次部署的目标终端
这次动作影响什么范围？	选中的终端（不是全教室）

这三个问题经常被混在一起。例如"终端部署页"——它的对象是"终端"还是"部署任务"？它是在改终端的长期配置，还是在设置本次部署的临时参数？这种含混在页面只有一个字段时不是问题，但在页面有 20 个字段时会导致用户完全分不清哪些是长期事实、哪些只是本次临时变化。

11.2 职责单一

一页只负责一个核心场景。两个场景混在一页，状态组合会指数增长，页面很快失控。

判断是否该拆页的标准：

适合拆成独立页	适合留在当前页
动作风险高（不可撤销）	只是当前对象的辅助查看
有明显的"确认前/确认后"分界	做完后用户继续在当前页工作
执行时间长，需要进度展示	本质上是小范围的参数调整
结果影响多个对象或多个页面	本质上是单个对象的确认

弹层和抽屉适用于：二次确认、局部补充输入、单对象验证。但如果弹层内承载的是一个完整流程，不应强行用弹层——这是常见的设计失误。

11.3 按钮必须有承接

页面上出现的每个按钮、链接或入口，都必须交代它的承接形式和结果：

```
按钮承接检查清单：
□ 点击后跳转到哪？（独立页 / 弹层 / 当前页状态切换 / 原地反馈）
□ 承接页面/弹层的职责是什么？
□ 完成后回到哪里？
□ 如果失败了呢？
```

如果一个按钮在文档或原型中出现了，但没有任何关于它承接什么的说明，这个设计就是不完整的。

12. 数据分层

12.1 四层数据模型

页面上不同性质的数据必须分层标识。实战中证明以下四层划分最为有效：

层次	含义	标识方式
当前实时状态	此刻终端/系统的真实情况	直接显示，通常带实时状态标记
本次任务参数	用户在本次任务中选择的设定	颜色区分或"→"箭头标记
已保存配置	系统中长期持久化的业务值	通常作为默认值或对比基准
外部系统数据	来自其他端或其他系统的信息	标注数据来源

这四层数据看起来都可以出现在同一页上，但含义完全不同：

- 实时状态回答"现在是什么情况"。
- 任务参数回答"这次准备怎么设"。
- 已保存配置回答"系统记住了什么"。
- 外部数据回答"其他地方知道什么"。

特别容易混淆的三组值：当前值、已保存配置值、本次准备写入的值。它们可以在页面上并排对比，但不能混成一个含糊的字段。

12.2 字段定义规范

一个关键字段的设计不只是给它起个名字。至少需要回答：

1. **它代表什么？** ——业务含义
2. **它从哪来？** ——数据来源
3. **它什么时候出现？** ——显示条件
4. **空值时怎么表达？** ——空态设计

5. **用户能凭它做什么判断？** ——设计意图
6. **用户不能凭它单独得出什么结论？** ——用来限制误用

最后两条尤其重要。很多页面问题不是缺字段，而是字段被滥用——一个只用于辅助验证的字段被设计成了唯一依据；一个只表示当前状态的字段被用户误认为长期配置。

13. 状态完备性

13.1 六态检查

每个页面至少需要主动检查以下六种状态：

状态	设计要点
默认态	用户进入时看到什么？
空态	没有数据是因为还没开始、前置条件不满足，还是真的没有？三种空态需要不同的文案和引导。
进行中	系统在做什么、用户在等什么、能否中途修改？
成功态	完成后用户最常做的下一个动作是什么？
失败态	哪里失败了？用户接下来应该重试、修改还是退出？
部分成功	哪些完成了、哪些没完成？页面焦点应该放在哪里？

部分成功尤其不能敷衍。 在批量任务中，30 台终端中 28 台成功、2 台失败——这个状态需要同时保留成功列表、失败列表和未处理列表。单一的"28/30 已完成"数字远远不够。

13.2 终端列表的特殊状态

在涉及终端（设备）列表的页面中，需要额外考虑以下状态维度：

维度	状态组合
在线状态	在线 / 离线
选择状态	已勾选 / 未勾选 / 不可选
执行状态	排队中 / 执行中 / 已完成 / 失败
角色	母机 / 普通终端 / 教师终端

这些维度相互正交，组合后的状态数量远大于直觉想象。最容易遗漏的是"离线但被选中""失败后等待重试""保持现状但需要显示沿用了什么"等交叉状态。

14. 流程设计

14.1 流程的七个环节

一个完整的任务流程应至少覆盖以下环节：

```
1. 对象/范围选择    → 当前处理什么？选了哪些？
2. 前置校验         → 是否满足执行条件？不满足怎么提示？
3. 参数设置         → 本次任务的配置项
4. 确认预览         → 执行前的最终确认，展示全部选项和预期结果
5. 执行与进度       → 分阶段展示进度、支持暂停/取消
6. 结果与处置       → 成功/失败/部分成功，以及对应的下一步
7. 善后与回溯       → 回到哪里？关联页面有什么变化？
```

不是每个流程都需要全部 7 个环节，但对于高风险、长执行时间或批量操作的任务，这 7 个环节通常都不能省略。

14.2 跨步骤一致性

同一任务的不同步骤之间，用户看到的数据必须一致：

- 步骤 1 选了 10 台终端，步骤 3 的确认页必须显示这 10 台。
- 步骤 2 设了某个参数，步骤 4 的进度页的标题/摘要应该反映这个参数。
- 步骤 5 的结果页应该可以追溯到步骤 1 的选择。

这种跨步骤一致性在原型中非常容易丢失，因为每个步骤通常由不同时间的不同工作轮次实现。

14.3 相似流程的逻辑一致性

如果产品中有多个相似流程（如"教室维护"、"考试桌面"、"重新部署"都涉及"选择桌面→选择终端→确认→执行"），**这些流程的骨架结构必须一致**：

- 相同环节使用相同的交互模式（选桌面的方式一样、选终端的方式一样）。
- 相同的字段使用相同的名称和格式。
- 相同操作使用相同的按钮文案。
- 桌面选择步骤如果在一个流程里支持设置默认启动/隐藏/还原模式，则在另一个流程里也应支持。

这是实战中最容易犯也最晚被发现的错误。 每个流程单独看都合理，但三个流程放在一起时，用户会发现同一类操作在三个地方有三种不同的交互方式——这对产品体验是致命的。

15. 输出与质量门禁

输出：

- 页面信息架构总纲（所有页面的清单、跨页关系、互斥关系）
- 各页面设计文档（每页含：场景说明、数据来源、字段定义、状态列表、按钮承接）

门禁：

1. 每个页面在信息架构总纲里有位置。
2. 每个按钮点击后的去向在文档中有明确交代。
3. 没有孤立页面——每页至少有一个入口和一个出口。
4. 所有相似流程的骨架结构一致。

第五部分　阶段四：可交互原型实现

> 本部分以一个基于 Node.js + Express + SSE 的多端联动原型为实例，描述从技术标准到代码组织的实现策略。这套方案在需要多端同屏、场景即时切换和一键复位的复杂原型中被充分验证过。如果你的原型较简单（几个静态页面、无需多端联动），完全可以用更轻量的方式——但本部分阐述的工程原则（状态集中、场景可控、增量迭代、修改即验证）对任何规模的可交互原型都成立。

16. 原型的技术标准

原型不是产品代码，但根据产品复杂度，需要考虑以下标准：

标准	含义	适用场景
可运行	评审方能看到真实的页面和交互	所有原型
可操控	现场能切换场景、触发动作、模拟异常	有多场景或异常分支的原型
可复位	能一键恢复到初始状态	需要多次演示或多轮评审的原型
多视角同屏	多端产品能同时展示联动效果	涉及多角色或多设备协同的产品

前两项几乎是所有可交互原型的刚需。后两项取决于产品复杂度——一个只有三个页面的单端原型不需要导演台和多视角同屏，但一个涉及管理端+终端+多种场景的产品原型，四项都不可少。

17. 架构策略

无论技术栈如何选择，可交互原型的架构核心是三条原则：**状态集中**（所有视图读写同一份数据）、**场景可控**（能快速切换到目标场景而不必从头操作）、**一键复位**（评审中途出问题能立即恢复）。

以下是一套在复杂多端原型中被验证有效的具体方案。

17.1 共享状态源

核心设计原则：**所有页面共享同一份服务端状态，状态变更集中处理，前端只负责渲染。**

```
┌──────────┐    SSE/WebSocket    ┌──────────┐
│ 平台端   │◄──────────────────►│          │
│ terminal │◄──────────────────►│ 服务端   │
│ 导演台   │◄──────────────────►│(state.json)│
└──────────┘                     └──────────┘
```

好处：

- 任意一端的操作都能实时反映到其他端。
- 不存在"这台终端看到的数据和那台不一样"的问题。
- 状态完全集中，便于回溯和复位。

对于不需要多端联动的单页原型，同样的原则可以用更轻量的方式实现——前端框架的全局状态管理（如 React Context、Vue Store）或者简单的 URL 参数驱动，同样能做到状态集中和场景切换。关键不在于用什么技术，而在于**只有一个状态真相源，且所有视图从这个真相源派生**。

17.2 导演台模式

为评审专门设计一个控制页面（导演台），用来：

- 切换场景（从"初始状态"切到"部署进行中"切到"部分失败"）。
- 触发动作（模拟"终端上线""终端失败""考试结束"）。
- 一键复位到初始状态。
- 显示当前系统状态的全景视图。
- 提供"推荐下一步"引导，帮助演示者按正确顺序执行。

导演台是产品说明和测试工具，不是产品页面。 产品页面上不应该出现仅供演示的指导性文案。

导演台的必要性取决于原型的复杂度。判断标准很简单：如果你在评审现场需要手动改代码、编辑配置文件或操作数据库来切换场景，你就需要导演台。如果所有场景都可以通过正常的产品操作自然到达，导演台就是过度设计。

17.3 种子数据

预先准备覆盖所有主要场景的种子数据：

```
种子数据覆盖清单：
□ 正常态（全部在线、数据完整）
□ 部分异常（有离线终端、有过期配置）
□ 初始态（空教室、未配置的新终端）
□ 进行中态（某个批量任务执行到一半）
□ 失败态（任务失败、部分终端失败）
□ 多教室（同一套系统管理多个教室）
```

绝不在评审现场临时造数据。 临时造出来的数据覆盖不了边界状态，且容易因为操作失误让演示卡住。

18. 代码组织

代码组织的核心原则是分离关注点：状态管理、视图渲染、共享工具和样式应该有清晰的目录边界。以下是一个基于 Node.js + Express 的多端原型项目结构示例——你的技术栈不同时目录会不同，但分离原则不变。

18.1 目录结构

```
原型项目/
├── server.mjs              # 服务端：状态管理 + API + SSE
├── data/
│   └── state.json          # 共享状态文件
├── public/
│   ├── apps/
│   │   ├── platform/       # 平台端页面
│   │   ├── terminal/       # 终端端页面
│   │   └── director/       # 导演台
│   └── shared/             # 共享模块
│       ├── helpers.js       # 数据查询函数
│       ├── ui.js            # UI 工具函数
│       └── vars.css         # CSS 变量（色彩体系）
├── package.json
└── README.md               # 启动说明
```

18.2 编码规范

规则	理由
CSS 颜色全部用 CSS 变量	统一色彩体系，支持主题切换
前端只负责渲染，不修改状态	状态变更集中在服务端，避免不一致
事件处理器和数据属性一一对应	方便审查"哪些交互是活的、哪些是断的"
产品页不含演示说明文案	导演说明只放导演台

19. 增量迭代策略

19.1 迭代顺序

```
1. 先搭全流程骨架（每页只有标题和主要按钮）
2. 逐页填充正常态内容
3. 补充异常态和部分成功态
4. 完善视觉细节和交互反馈
```

关键：每轮迭代结束后，必须能从头到尾跑通已实现的流程。不能出现"A 页完美但 B 页还是空的所以流程断了"的情况。

19.2 每次修改的安全流程

```
修改 → 语法检查 → 影响面扫描 → 全流程回归
```

尤其在 AI 辅助编码时，"修改"和"全流程回归"之间不能省任何步骤。AI 修改某处代码时，最容易遗漏的是事件处理器绑定、数据属性名一致性和跨页面状态同步。

20. 输出与质量门禁

输出：可运行的原型项目 + 种子数据 + 导演台

门禁：

1. 一条命令即可启动（如 `npm start`）。
2. 导演台能覆盖所有主要场景。
3. 所有页面数据来自共享状态，不存在硬编码假数据。
4. 全流程可从头走到尾。

第六部分　阶段五：评审与修正

21. 评审准备

1. **编排走查清单**：按"核心流程 → 分支流程 → 异常场景"的顺序组织。
2. **验证种子数据**：确认每个走查项都有对应的种子数据。
3. **完整预演**：正式评审前至少预演一遍，确保没有流程断裂。

22. 评审中的记录方法

评审方反馈需要立即分类：

类型	定义	优先级
事实层修正	业务规则/对象/关系需要变更	最高——影响所有相关页面和代码
结构层修正	页面关系/流程顺序需要调整	高——影响多个页面
表现层修正	字段措辞/布局/视觉微调	中——影响单个页面

23. 评审后修正的分层执行

永远按照"从底层到表层"的顺序修正：

```
修事实文档 → 修页面设计 → 修原型代码 → 同步记忆
```

具体展开：

1. 先修改事实层文档（正式基线、稳定结论）。
2. 再修改页面设计文档（信息架构、单页设计）。
3. 然后修改原型代码。
4. 最后同步更新仓库记忆和事实索引。

绝不能反过来。 先改了代码再补文档，文档会失真，后续工作会在失真文档上展开。

第七部分　高级工程实践

24. 跨页一致性管理

24.1 术语一致性

同一业务字段在不同页面上必须使用相同的标签名、数据来源和格式。

病态案例：同一字段在 A 页叫"终端类型"、B 页叫"终端用途"、C 页叫"角色"——评审方不会认为它们是同一字段。

治理方式：

1. 定义术语表，入正式基线。
2. 区分视角词（用户看到的标签）、动作词（系统操作）、存储词（数据库字段名），三者可以不同但关系必须明确。
3. 每次产出文档或代码后做术语扫描，确保无旧术语残留。

24.2 交互模式一致性

相似页面使用相同的交互模式：

检查项	要求
桌面选择	所有流程的桌面选择步骤使用统一的卡片选择组件和内联设置项
终端选择	所有流程的终端选择列表使用统一的列定义、全选/清空按钮和排序功能
确认页	所有流程的确认页展示相同级别的详情（桌面配置 + 终端列表）
进度页	所有流程的进度页使用统一的进度展示组件

24.3 数据口径一致性

同一份数据在不同页面上的展示必须一致：

- 终端在选择页勾选了 10 台，确认页必须只列出这 10 台。
- 桌面在选择步骤设的还原模式是"还原系统盘"，确认页的汇总里必须显示相同的值。
- A 页面通过"设为默认启动"按钮设置了默认桌面，B 页面的桌面列表里该桌面必须带有"默认启动"标记。

25. 多端协同设计

当产品涉及多端时（如管理平台 + 终端），需要额外处理：

25.1 同一时刻，各端各在什么页面

例如：母机正在执行部署时，受控终端应该停留在"等待部署写入"页面，平台端应该能看到部署进度。这种"同一时刻各端各在哪"的全景视图，应该在信息架构总纲中明确。

25.2 互斥关系

某些操作在某一端进行时，其他端不能进行某些操作：

```
- 母机正在执行部署任务时，终端侧不能单独修改 IP。
- 考试模式激活期间，终端侧的桌面管理功能受限。
```

25.3 状态同步延迟

原型中通常假设状态同步是实时的（通过 SSE），但在评审讲解时应该能说明：实际产品中如果同步延迟了会怎样。

26. AI 辅助原型实现的注意事项

让 AI 帮你写原型代码，和让 AI 帮你写一个函数，是完全不同的事。函数的上下文在一个文件内，原型的上下文散落在状态模型、多个页面模板、事件处理器、CSS 变量和种子数据之间。AI 每一轮只能看到其中一部分，而原型的正确性取决于这些部分之间的一致性。

以下是实战中反复出现的问题。它们不是某个项目的特例，而是 AI 辅助原型开发中的结构性缺陷——只要你用 AI 写超过三个页面的可交互原型，大概率会遇到其中大部分。

26.1 只生成成功态

AI 天然倾向于生成"最完整、最正常"的页面——数据都在、状态都对、每个字段都有值。但评审现场被问到的第一个问题往往是"如果这里没有数据呢？""如果这步失败了呢？"

AI 不会主动想到空态、错误态和部分成功态，除非你在每个页面的需求描述里显式要求。即便你要求了，AI 生成空态时也倾向于放一句"暂无数据"文案了事，而不是区分"还没开始""前置条件不满足""真的没有"这三种不同含义的空。

26.2 事件绑定断裂

这是最高频的代码级 bug。AI 在 HTML 模板里写了 `data-some-action="xxx"` 属性，但 JavaScript 中没有对应的处理器；或者处理器存在但属性名拼错了一个字母。页面看起来完全正常，按钮也在，点了没反应——在演示现场才发现。

更隐蔽的变体是：AI 修改了某个按钮的属性名来修复一个 bug，但忘了同步修改处理器中的选择器。或者反过来，处理器改了但模板没改。这类问题只能通过系统性地对比"模板中的所有交互属性"和"处理器中的所有选择器"来发现。

26.3 跨页状态泄漏

AI 实现了"在 A 页面保存滚动位置，刷新后恢复"的功能——但没有意识到这个保存是全局的。用户在 A 页滚到底部，切换到 B 页，B 页也从底部开始显示。类似的泄漏还有：表单填了一半切走再切回来，数据消失了或保留了不该保留的旧值；选中状态跨页面持续存在等。

根本原因是 AI 把"页面内的瞬态"当成了"全局持久态"来处理。纠正的原则是：**瞬态只在对应的页面/组件生命周期内有效，切换上下文时必须重置。**

26.4 相似流程各写一套

三个看起来相似的流程（比如教室维护、考试桌面、重新部署的桌面选择步骤），AI 会为每个流程独立实现一套代码。表面上每个都能用，但用的是三种不同的属性名、三种不同的交互模式、三种不同的排版。改一个流程的交互时，其他两个不会自动跟着改。

这不仅是代码维护问题，更是产品体验问题——评审方会发现同一类操作在三个地方有三种不同的做法。

26.5 共享状态 schema 漂移

原型通常有一个共享的状态模型（如 JSON 文件）。AI 在实现某个新功能时，会直接在状态模型里加字段——但不会检查其他页面、其他端是否也需要感知这个字段。结果：A 页面往状态里写了一个新字段，B 页面完全不知道这个字段存在，C 页面用了一个含义相近但名字不同的字段。

更糟的情况是 AI 在前端代码里"假设"了某个状态字段存在，但服务端从未定义它——一切正常直到真的有数据流过这条路径。

26.6 样式硬编码

AI 倾向于给每个元素写具体的颜色值、像素数和字号，而不是引用设计系统中已有的变量。结果：你想改主色调，发现需要搜索替换 47 处 `#1890ff`；你想统一间距，发现每个页面用的都是不同的 `padding` 值。

更微妙的问题是 AI 会让不同页面上"看起来像同一种组件"的元素使用完全不同的 CSS 实现，导致后续统一调整时根本找不全需要改的地方。

26.7 幻影交互

AI 生成的页面上有按钮、有链接、有下拉框——看起来功能完备。但其中有些只是 HTML 元素，背后没有任何逻辑。它们之所以出现，是因为 AI 认为"这个页面应该有这些功能"，但实际开发时只实现了一部分。

这类"幻影交互"在静态截图审查中无法发现，只有在真正点击时才暴露。评审现场发现一个断掉的按钮，评审方对整个原型的信任度会立即下降。

26.8 复制粘贴导致的隐性差异

AI 从一个已有页面复制组件到新页面时，经常做"微调"——改几个标签名、改个默认值、调一下排列顺序。这些微调孤立来看都合理，但积累起来会导致两个"本应相同"的组件出现令人困惑的差异。

更危险的是：AI 复制了处理器代码但忘了改里面的硬编码 ID 或页面名，导致 B 页面的按钮实际触发的是 A 页面的逻辑。

26.9 过度工程化

AI 有时会为一个简单需求构建复杂的抽象层——为只出现一次的组件写通用工厂函数，为三行配置写一套解析引擎。这种过度工程化让原型变得难以理解和修改，下一轮迭代时连 AI 自己都可能读不懂上一轮写的抽象。

原型的生命周期很短，迭代速度很快。**在原型中，可读性和可改性的优先级远高于可复用性。**

26.10 不验证就交付

AI 完成一批修改后，如果你不要求它验证，它默认认为"改完即正确"。但原型是多文件耦合的系统——改了模板可能漏了处理器，改了状态模型可能漏了渲染逻辑，改了一个流程可能破坏了相似流程的一致性。

每次修改后的强制验证流程：语法检查 → 交互属性与处理器一一对应 → 受影响的全部流程从头走到尾 → 相似页面的一致性复查。省掉任何一步，都是在给评审现场埋雷。

27. 持续验证

原型不是一次性产物，它会随着需求变更和评审反馈持续迭代。持续验证的关键实践：

实践	方式
自动化冒烟测试	用脚本验证核心流程可走通
种子数据覆盖度检查	确保每个场景都有对应的测试数据
状态一致性校验	自动检查所有页面引用的数据字段是否在状态对象中存在
按钮承接完整性	自动扫描 HTML 中的交互属性，检查是否都有对应处理器

第八部分　评审检查清单

无论你是原型的制作者还是评审者，以下问题帮助你系统性地检查原型质量：

28. 页面级检查

1. **这页的主对象是什么？** 先确认中心，否则后续讨论都会脱焦。
2. **用户从哪来、到哪去？** 前后链路不清楚的页面是孤岛。
3. **当前页改的是长期配置还是本次任务参数？** 如果搞混了，用户会以为临时操作永久生效。
4. **关键字段从哪来？** 字段来源不清，后续无法判断它显示的是当前值、历史值还是任务临时值。
5. **有没有未决机制被写成了已定行为？** 原型最容易越权的地方，尤其在 AI 辅助生成时。
6. **非理想状态处理完整吗？** 空态、失败态、部分成功态是否都有设计。
7. **同一对象在标题、正文、按钮里是否一致？** 尤其注意视角词、动作词、存储词的混用。

29. 流程级检查

1. **全流程能否从头走到尾？** 包括正常流程和主要异常分支。
2. **相似流程的骨架结构是否一致？** 这是最常被忽视但影响最大的问题。
3. **跨步骤的数据是否一致？** 步骤 1 选的终端 = 步骤 3 确认页的终端 = 步骤 5 结果页的终端。
4. **每个按钮都有承接吗？** 没有承接的按钮是断裂点。
5. **技术限制是否已表达在页面上？** 那些会影响用户选择的技术约束，不能只存在于研发文档中。

30. 系统级检查

1. **多端同一时刻各自在什么页面？** 是否有全景视图。
2. **状态同步是否正确？** A 端操作后 B 端是否正确反映变化。
3. **数据口径是否全局一致？** 同一字段在不同页面的名称、来源、格式是否统一。
4. **导演台能否覆盖所有评审场景？** 是否需要在现场临时造数据——如果需要，说明种子数据不够。

附录

A. 页面设计文档模板

```
# [页面编号] [页面名称]

## 场景
- 谁来到这页、在什么情况下来到
- 来到时最想先确认什么
- 做完后的下一步

## 数据来源
- 当前实时状态：[来源]
- 本次任务参数：[来源]
- 已保存配置：[来源]
- 外部系统数据：[来源]

## 字段定义
| 字段名 | 数据层 | 来源 | 空值表达 | 设计意图 |
| :--- | :--- | :--- | :--- | :--- |

## 按钮承接
| 按钮 | 承接形式 | 描述 |
| :--- | :--- | :--- |

## 状态列表
| 状态 | 描述 | 触发条件 |
| :--- | :--- | :--- |

## 与其他页面的关系
- 入口：从 [页面] 进入
- 出口：完成后回到 [页面]
- 互斥：与 [页面] 不能同时存在
```

B. 评审走查清单模板

```
# 评审走查清单

## 核心流程
- [ ] 场景1：[描述] → 预期结果：[描述]
- [ ] 场景2：[描述] → 预期结果：[描述]

## 分支流程
- [ ] 分支1：[触发条件] → 预期行为：[描述]

## 异常场景
- [ ] 异常1：[触发方式] → 预期表现：[描述]

## 多端联动
- [ ] A端做[操作]后，B端应显示[变化]
```

C. 修改安全检查清单

```
□ 影响面审计完成（列出所有受影响的文件/函数/属性）
□ 修改已完成
□ 语法/编译验证通过
□ 事件绑定与 HTML 属性一一对应
□ 数据属性名在所有引用位置一致
□ 术语一致性检查通过
□ 全流程回归通过（不只是修改点）
□ 相似页面的一致性检查通过
□ 文档与代码之间无矛盾
```